מטלה 4 – מימוש תהליכי ליבה, פיסיקה וארכיטקטורה

ממשו את *תהליכי הליבה* של N+1 הרעיונות המקוריים שלכם.

מימוש תהליכי הליבה

להזכירכם: תהליך הליבה הוא התהליך העיקרי שהשחקן מפעיל כדי להתקדם במשחק. דגשים:

- בשלב זה *אין צורך* לממש את העיצוב הגרפי הסופי של המשחק – אפשר להשתמש בצורות פשוטות (העיקר שנבין למה התכוונתם).
- גם אם המשחק שלכם מתוכנן לשלושה ממדים, בשלב זה מספיק לממש את התהליך בשני ממדים.
- גם אם במשחק שלכם יש כמה רמות, בשלב זה מספיק לממש רמה אחת.
- מטרת המטלה היא להמחיש את ה"משחקיות" - מה בדיוק עושים השחקנים במשחק שלכם.
- בעקבות המשוב שתקבלו על המטלה, ייתכן שהמשחק ישתנה באופן משמעותי, אז אל "תיקשרו" לפרויקט יותר מדי – זה מקובל לכתוב משחקים מחדש בעקבות משוב - זה חלק מתהליך הלמידה.
- במימוש המשחקים, יש להשתמש ב*פיסיקה*, וכן להקפיד על *ארכיטקטורת פרויקט* – ראו הנחיות בעמודים הבאים.

א. פיסיקה

יש להשתמש לפחות פעם אחת (לפחות באחד מהמשחקים שלכם) בכל אחת מהתכונות הפיסיקליות שנלמדו בשיעור:

- בקר דמות קינמטי – CharacterController;
- גוף קשיח פיסיקלי – Rigidbody או Rigidbody2D.
- כוחות – AddForce או AddTorque.
- הטלת קרניים / כדורים – Raycast או SphereCast.
- בדיקת התנגשויות והתכונות של עצם ההתנגשות - Collision או Collision2D .
- רשות – אפקטורים (Effector) או מחברים (Joint), לבניית סביבות ומכונות פיסיקליות מעניינות.

ב. ארכיטקטורת פרויקט

ארגנו את הקוד במשחק שלכם כך שישקף עקרונות של ארכיטקטורת קוד נכונה. המטרה היא, שאפשר יהיה לשנות ולהרחיב את המשחק בקלות, בלי לשבור אותו ובלי לשנות המון קוד בבת אחת.

1. ארגון הסקריפטים בתיקיות עם שמות הגיוניים, לפי תחום אחריות (שחקן, אויבים, ממשק וכו'), ולא תיקייה אחת של "מנהלים". דוגמה למבנה אפשרי:

```
Assets/
├── Scripts/
│   ├── Player/        PlayerMovement.cs, PlayerShooter.cs
│   ├── Enemies/       EnemyChase.cs, EnemySpawner.cs
│   ├── Combat/        Health.cs, DamageDealer.cs, IDamageable.cs
│   ├── Collectibles/  Collectible.cs
│   ├── Game/          RoundRules.cs, ScoreKeeper.cs, RoundTimer.cs
│   ├── UI/            ScoreLabel.cs, LivesLabel.cs, EndScreen.cs
│   └── Events/        GameEvent.cs  (ScriptableObject)
├── Data/              EnemyStats.asset, LevelSettings.asset
├── Prefabs/
└── Scenes/
```

2. שמירה על קוד נקי (**Clean Code**):
   - כל מחלקה צריכה להיות בעלת אחריות ממוקדת (עקרון Single Responsibility). מבחן פשוט: אפשר לתאר את האחריות של כל מחלקה במשפט אחד, בלי המילה "וגם".
   - שמות משתנים ופונקציות צריכים להיות תיאוריים וברורים.
   - הימנעו ממחלקות ענקיות. העדיפו כמה רכיבים קטנים על אותו אובייקט (למשל Health, Mover, DamageDealer) על פני מחלקה אחת שעושה הכול.
   - מומלץ לכתוב הערות קצרות (Comments) רק היכן שהדבר תורם להבנה, ולא באופן מוגזם.
3. תקשורת בין רכיבים:
   - לפחות מקום אחד שבו רכיב מודיע על משהו דרך אירוע (event, UnityEvent או ScriptableObject), במקום לקרוא ישירות למחלקה אחרת.
   - קוד המשחק לא מכיר את קוד ממשק-המשתמש: הממשק מאזין לאירועים של המשחק, ולא להפך.
   - כל הרשמה לאירוע ב-OnEnable מבוטלת ב-.OnDisable
   - סינגלטון או משתנה סטטי: רק עם הצדקה ברידמי.
4. נתונים בנפרד מהלוגיקה:
   - ערכים לכוונון (מהירות, חיים, ניקוד, זמנים) נשמרים ב-ScriptableObject או בשדות Inspector, ולא כמספרים קבועים שמפוזרים בקוד.

בהצגה תתבקשו להסביר איך הייתם מבצעים שינוי קטן במשחק (למשל סוג אויב חדש): אילו קבצים משתנים, ולמה רק הם צריכים להשתנות.

תהליך עבודה (Workflow):

1. תכנון מבנה המחלקות:
   - צרו תרשים UML בסיסי או שרטוט על דף שמראה איך המחלקות מתקשרות זו עם זו.

- החליטו איך **המשחק** יידע לעדכן את הממשק כאשר השחקן אוסף חפץ, וכו'. למשל דרך אירוע (ראו "תקשורת בין רכיבים").

2. מימוש בסיסי של תהליך הליבה של המשחק, כפי שהוסבר בתחילת הקובץ.
3. השלמת לולאת המשחק:
   - ודאו שהתנאי לניצחון/הפסד פועל: אם הזמן נגמר, או אם אספתם מספיק חפצים וכו', מגיעים למסך ההפסד (Game Over) או למסך ניצחון (Win Screen) לפי המצב.
4. **Refactor** - שיפור מבנה הקוד:
   - אם יש קוד שחוזר על עצמו (כמו אלגוריתם תנועה לאויבים שונים), שקלו להשתמש במחלקת אב משותפת או ממשק (Interface). עדיף להתחיל מהרכבה: רכיב קטן ומשותף (למשל Mover) שכל סוגי האויבים משתמשים בו; ממשק קטן משותף (כמו Damageable) לכמה סוגי עצמים שמקבלים נזק.
   - הקפידו שכל סקריפט נשאר ממוקד יחסית ואינו מתנפח ליותר מדי פונקציות ותכונות.
   - כל שינוי משמעותי נכנס דרך Pull Request, שחבר אחר בצוות קורא ומאשר.
   - בקשו מסוכן AI לבדוק את הקוד מול התרשים, ותעדו ביומן השימוש איפה הקוד סטה מהתרשים ולמה.

הגשה

1. ריפוזיטורי מסודר ב **GitHub**
2. **README** קצר שמסביר:
   - איך מריצים את המשחק.
   - תיאור קצר של יחסי המחלקות (Class Relationships). יש לשים תרשים מחלקות (UML) או תרשים בסיסי אחר, שמציג את המחלקות, הקשרים החשובים ביניהן, האירועים, מי שולח אותם, ומי מקבל אותם. **שימו לב:** אפשר לשים תרשימים ישירות ברידמי בעזרת mermaid. יש הסבר בתיעוד של גיטהאב: https://docs.github.com/en/get-started/writing-on-github/working-with-advanced-formatting/creating-diagrams
   - 2-3 החלטות ארכיטקטורה: מה בחרתם, מה הייתה החלופה, ולמה (למשל: אירוע במקום הפניה ישירה מהשחקן לממשק).
   - איך מוסיפים סוג אויב או חפץ חדש: אילו קבצים משתנים.
   - אם השתמשתם בסינגלטון או במשתנה סטטי: איפה, ולמה.
3. בניה והעלאת המשחק לItch.io בפורמט Web עם פירוט הוראות איך לשחק ולינקים לריפוזיטורי מItch ומהItch לריפוזיטורי.